



SCHOOL OF COMPUTER SCIENCE

April 2025 Semester

Computer Vision and Natural Language Processing

(ITS69204)

Group Assignment (30%)

Date: 18th July, 2025

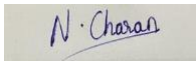
<u>No.</u>	<u>Student Name</u>	<u>Student ID</u>	<u>Signature</u>	<u>Score</u>
1.	Charan Teja Nagisettygari	0371148		
2.	Hammad Ruxshan	0356865	hammad	
3.	Teoh Nian Hao	0364281	teoh	
4.	Yang Jintao	0371701		
5.	Kashfa Mirza Bhasha	0336176		

Table of Contents

Introduction -	3
Dataset Description -	3
Dataset Composition -	3
Challenges and Limitations -	4
Aim -	4
Objectives -	4
Motivation and Significance -	5
Potential Applications -	5
Methodology	6
Architecture Design -	6
Comparison with Alternative Architectures -	8
Implementation and Code Documentation -	8
load_dataset() function -	9
CNN model -	10
Augmentation Generators -	13
Training callbacks -	15
Training	17
Data splitting -	17
Detailed Split Analysis -	18
Sample Image Visualisation -	18
Data Leakage Detection -	19
Model Training -	20
Evaluation and Metrics	21
Introduction to Model Evaluation -	21
Confusion Matrix Interpretation -	22
Model Training Behaviour and Overfitting Analysis -	24
Interpretation of Strengths and Weaknesses -	24
Conclusion	25
References	26

Introduction -

Potatoes are a globally significant crop, serving as an important food source for millions all around the world and also contributing substantially to the agricultural economy. However, farming potatoes face numerous challenges, with leaf diseases being among the most harmful (Regina, 2022).

Potato leaf diseases are amongst the most common and damaging threats to potato crops. These diseases can be caused by bacteria, fungi, viruses, pests, nematodes, or pathogens such as *Phytophthora* (PlantVillage, n.d.-a). If not detected and treated promptly, they can lead to severe crop damage, reduced yield, and economic loss for farmers (Li et al., 2025). Early and accurate identification of disease types is essential for applying the right treatment methods and preventing the spread of infection so that the potato plants can be protected. (Regina, 2022).

Traditionally, farmers or agricultural experts detect these diseases through visual inspection of the leaves. While this method can be effective, it is time-consuming and subject to human error, especially when symptoms are subtle or when multiple diseases appear similar (PlantVillage, n.d.-a). In rural or under-resourced areas, there may not be experts available to perform accurate diagnosis on the potato leaves, leading to delayed or incorrect treatment and causing the conditions of the potatoes to worsen.

Dataset Description -

The dataset that we are using for this project is titled “Potato Plant Diseases Data”, curated by Hafiz Nouman and publicly available on Kaggle. This dataset is specifically designed for the classification of potato plant diseases using image data. It provides high-resolution images of potato plants exhibiting various conditions, including Early Blight, Late Blight, and Healthy Leaves. We intend to use it in training machine learning and deep learning models for our CNN.

Dataset Composition -

The dataset contains images of potato leaves, labelled Early Blight, Late Blight, and Healthy Leaves.

Table 1 - Dataset Composition

Total Number of Classes	Image Format	Colour Mode	Dimensions	Images per class
3	JPG	RGB (colour images)	256x256	Early Blight - 1000 Late Blight - 1000 Healthy - 156

Challenges and Limitations -

- **Class Imbalance Challenge** - Extreme 6.58:1 imbalance (152 healthy, 1000 of each disease) makes the models biased towards the disease classes, but it corresponds to the situation the model is intended to represent in the low-level agricultural world.
- **Limits to Size of the Dataset** - The collection of images (2,152 ones, only 152 healthy) is not large enough to provide the widest variety of patterns to learn and is vulnerable to overfitting challenges of robust deep learning.
- **Complexity of the Disease** - The ability to distinguish subtle differences between early/late blights, lack of stages of disease progression and co-infections restricts field use.
- **Generalization Limitations** - Limitation in the geography and lack of seasonal/cultivar variation constrains the ability of global agricultural deployment to diverse environments.
- **Technical Limitations** - Fixed resolution RGB images do not contain multispectral data and time series information that is important to provide a detailed monitoring of diseases.

Aim -

The project proposed to design and test a CNN-based solution that will automatically classify potato leaf diseases on RGB images with the focus on their correct detection and classification into such categories as healthy, bacteria, fungi, virus, pest, nematode, and phytophthora (PlantVillage, n.d.-a; Regina, 2022).

Objectives -

To achieve this goal, we outlined specific objectives:

- Preprocess and explore the Potato Leaf Disease dataset, making sure it is suitable in terms of data to train a deep learning model.
- Develop a CNN network that could learn spatial and visual characteristics with potato leaves (Jorgecardete, 2024).
- Train CNN and test its performance on such metrics as accuracy, precision, recall, and F1-score.
- Evaluate model capabilities and their ability to be used in the context of real applications in the agricultural sector, especially in the case of remote locations (Li et al., 2025).
- Define the hardships in the project and propose future enhancement.

Motivation and Significance -

This project brings value to the AI-assisted agriculture research and at the same time gives experience of computer vision implementation on real-world farming data. The CNN method can be used to learn the advanced technique in data preprocessing and model building and interpreting the results (GeeksforGeeks, 2025; Jorgecardete, 2024), and it becomes pedagogically valuable and probably allows the real knowledge transition to the real world of farming.

Potential Applications -

When we successfully develop the CNN there can be a few ways that we could utilize in in real life scenarios.

- **Mobile Farming Apps** - A lightweight version of the model can be deployed in mobile applications. Farmers can take pictures of their plants using a smartphone, and the app can instantly diagnose the disease type and suggest treatment methods.
- **Early Disease Detection and Crop Monitoring** - Real-time analysis of leaf health can help detect diseases in their earliest stages. This can significantly reduce crop losses and improve overall productivity.
- **Automated Drones and Field Cameras** - Integrated with drones or stationary field cameras, the model can scan entire fields for signs of disease. This allows large-scale

farms to automate field surveillance and monitor crop health without manual labor.

- **Agricultural Extension Services** - Governments and NGOs can use this technology to assist smallholder farmers by providing affordable disease diagnosis tools and reducing reliance on in-person inspections.

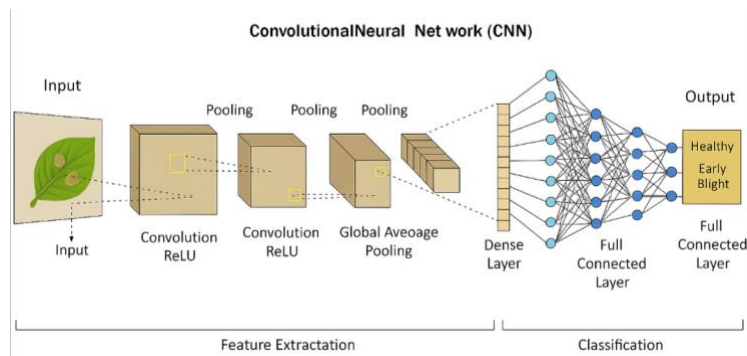
Methodology

Architecture Design -

Table 2 - Architecture Design for the model

Layer Type	Configuration	Justification
Input Layer	200 x 200 x 3 RGB image	Balances visual detail and computational efficiency
		RGB channels retain color features important for detecting diseases.
Convolution Layer 1	32 filters, 3x3 kernel	Increasing filters at deeper layers captures more complex patterns. Start small to reduce computation in early layers. Kernel size is common and efficient for capturing local texture features. Ideal for detecting small disease spots and edges.
Convolution Layer 2	64 filters, 3x3 kernel	
Convolution Layer 3	128 filters, 3x3 kernel	
MaxPooling Layer 1	2x2 pooling	Reduces spatial dimensions and computation.
MaxPooling Layer 2	2x2 pooling	

MaxPooling Layer 3	2x2 pooling	Helps focus on dominant features and adds robustness to shifts.
Flatten Layer	Flattens 3D feature maps into 1D vector	Converts 3D feature maps to 1D vector for dense layer compatibility while preserving spatial information.
Dense Layer	512 Units	Provides enough capacity to learn complex features.
		Balanced to avoid unnecessary complexity.
Dropout Layer	Dropout rate = 0.5	Prevents overfitting by randomly disabling neurons during training.
		Encourages generalization.
Output Layer	3 units	Matches the 3-class classification problem (Healthy, Early Blight, Late Blight).
		Softmax outputs interpretable probability scores.



Comparison with Alternative Architectures -

Evaluation of model performance, complexity and appropriateness of datasets is important in agricultural image classification. On large datasets such as ImageNet, models such as MobileNet and ResNet work very well, but on a small dataset such as our potato disease data, they can be in practice useless. Pretrained models are computationally demanding and could not adapt to small data as there are not many variations. We have three distinct classes (Healthy, Early Blight, Late Blight) which can be identified by their visible symptoms (i.e. color, shape, and texture). Very deep networks such as ResNet can create unnecessary computing overhead, slow training, and overfitting since we have a small volume of data. The custom CNN comes with important benefits; reducing the number of layers and parameters means quicker training and one can deploy on low-resource devices such as farms or mobile devices. The architecture learns on relevant leaf features but does not overly complicate the learning process, has an interpretable structure and is fine-tuneable to enable insight into the predictions. Although pretrained networks are suitable in complex cases, our customized CNN is the best network to integrate in targeted potato disease classification since it is efficient, accurate, and suitable in real world agriculture where simplicity, speed, and reliability are ideal.

Implementation and Code Documentation -

This block of codes mainly imports all the libraries being used throughout the program.

```
# Import necessary libraries for deep learning and data processing
import kagglehub
import tensorflow as tf
from tensorflow.keras import layers, Model
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img, img_to_array
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os
import cv2
from collections import Counter
import warnings
warnings.filterwarnings('ignore')
```

Picture 1 - Code for importing libraries

- **Deep Learning Libraries** - The core libraries are kagglehub for downloading the dataset, tensorflow/keras for a deep learning framework and high-level API, layers/Model for a neural network architecture, ImageDataGenerator for data augmentation, load_img/img_to_array for image processing, Adam optimizer for training and EarlyStopping/ReduceLROnPlateau for training optimization.

- **Import Data Analysis Libraries** - The baseline class includes `split_train_test_split` for dividing dataset, `classification_report/confusion_matrix/accuracy_score` for model evaluation, `numpy` for numbers computation, and `pandas` for data manipulating.
- **Extra Libraries** - Utilities consists of sub-libraries such as `matplotlib.pyplot/seaborn` for visualisation of data, `os` for file system functions, `cv2` for image processing from OpenCV, `Counter` for handling the class labels and `warnings.filterwarnings('ignore')` for clean output printing.

load_dataset() function -

```
def load_dataset(data_path):
    """Load images and labels from the nested dataset directory structure"""
    images = []
    labels = []
    class_names = []

    # Navigate to PotatoPlants directory which contains the class folders
    potato_plants_path = os.path.join(data_path, 'PotatoPlants')

    if not os.path.exists(potato_plants_path):
        print(f"PotatoPlants directory not found at {potato_plants_path}")
        print("Available directories:", os.listdir(data_path))
        return np.array([]), np.array([]), []

    # Get class directories from PotatoPlants folder
    for class_dir in sorted(os.listdir(potato_plants_path)):
        class_path = os.path.join(potato_plants_path, class_dir)
        if os.path.isdir(class_path):
            class_names.append(class_dir)

    print(f"Found {len(class_names)} disease classes: {class_names}")

    # Load images from each class
    for idx, class_name in enumerate(class_names):
        class_path = os.path.join(potato_plants_path, class_name)
        image_files = [f for f in os.listdir(class_path) if f.lower().endswith(('.png', '.jpg', '.jpeg'))]

        print(f"Loading {len(image_files)} images from {class_name}")

        for image_file in image_files:
            image_path = os.path.join(class_path, image_file)
            try:
                # Load and resize image to 200x200 pixels
                image = cv2.imread(image_path)
                image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
                image = cv2.resize(image, (200, 200))
                image = image.astype('float32') / 255.0

                images.append(image)
                labels.append(idx)
            except Exception as e:
                print(f"Error loading {image_path}: {e}")

    return np.array(images), np.array(labels), class_names

# Load the complete dataset
X, y, class_names = load_dataset(path)
print(f"Dataset loaded: {X.shape[0]} images of shape {X.shape[1:]}")
print(f"Classes: {class_names}")

# Display class distribution to understand the imbalance
class_counts = Counter(y)
print("\nClass distribution:")
for i, class_name in enumerate(class_names):
    print(f"{class_name}: {class_counts[i]} images")
```

Picture 4 - Information about the dataset

Loading and preprocessing of the potato leaf image dataset in structured directory form is the role of the `load_dataset(data_path)` function. It starts with reading into a parent directory `PotatoPlants` which is the root to many folders that define each disease category (e.g. Early

Blight, Late Blight, Healthy). The purpose of the function is to check the directory structure, gathers the names of classes, and sequentially executes the loading of images into each class folder. It then reads in each of the images, applies the transformation on the BGR to RGB color space to RGB color space, resizes the image to 200 x 200 pixels and normalizes the pixel values to lie in the range [0, 1] and of type float32 to be compatible with a deep learning model. It takes care of the file filtering, so that only valid image formats (.jpg, .png, .jpeg) will be processed and it also includes an error handling to trap and log unreadable images. Lastly, it yields processed data of an image, labels and names of classes as NumPy arrays. The role ensures there is clean data ingestion which is vital in preserving reproducibility and consistency between experiments.

```
Found 3 disease classes: ['Potato__Early_blight', 'Potato__Late_blight', 'Potato__healthy']
Loading 1000 images from Potato__Early_blight
Loading 1000 images from Potato__Late_blight
Loading 152 images from Potato__healthy
Dataset loaded: 2152 images of shape (200, 200, 3)
Classes: ['Potato__Early_blight', 'Potato__Late_blight', 'Potato__healthy']

Class distribution:
Potato__Early_blight: 1000 images
Potato__Late_blight: 1000 images
Potato__healthy: 152 images
```

Picture 5 - Information about the dataset

The output shows that the dataset has three classes: Early blight (1000), Late blight(1000) and Healthy plants(152). There are a total of 2152 photos that were loaded with each photo resized to 200x200 pixels with 3 colour channels (RGB). The last class distribution detected a class imbalance which needs to be dealt with by using techniques like oversampling or class weights, to ensure it does not affect the model.

CNN model -

The following codes were used to create a CNN (convolutional neural network) model.

```
def create_potato_disease_cnn(input_shape=(200, 200, 3), num_classes=3):
    """
    Create CNN architecture for potato disease classification
    Uses 3 convolutional blocks followed by dense layers for classification
    """
```

Picture 6 - Code for defining CNN model

The code above is defining a function named “create_potato_disease_cnn()”, used to create and return a CNN model. It has two parameters: “input_shape” and “num_classes”. The first is set to (200, 200, 3) which represents the height, width and the number of channels for RGB photos. The second is set to 3 which represents the number of disease categories.

```
# Input layer
inputs = keras.Input(shape=input_shape, name='input_layer')
```

Picture 7 - Code for the input layer

The code snippet above creates the input layer for the CNN model using the Keras API. It helps to accept data from the image using the shape defined by “input_shape”.

```
# First convolutional block
x = layers.Conv2D(32, (3, 3), activation='relu', name='conv2d_block1')(inputs)
x = layers.MaxPooling2D((2, 2), name='maxpool_block1')(x)
```

Picture 8 - Code for the first hidden layer

The code above is for the first convolutional block. It has 32 filters with a 3x3 size and uses the ReLU activation function to help with non-linearity in the model. It also includes a max pooling layer with a pool size of 2x2 following the convolution to prevent overfitting by lessening the spatial dimensions of the feature map.

```
# Second convolutional block
x = layers.Conv2D(64, (3, 3), activation='relu', name='conv2d_block2')(x)
x = layers.MaxPooling2D((2, 2), name='maxpool_block2')(x)
```

Picture 9 - Code for second hidden layer

The code above is for the second convolution layer with 64 filters that have a 3x3 size. It helps to detect abstract features and is followed by a 2x2 maxpooling layer to allow more downsampling.

```
# Third convolutional block
x = layers.Conv2D(128, (3, 3), activation='relu', name='conv2d_block3')(x)
x = layers.MaxPooling2D((2, 2), name='maxpool_block3')(x)
```

Picture 10 - Code for third hidden layer

The code snippet above shows the third and final convolutional layer that has 128 3x3 filters to extract high-level semantic attributes like patterns for specific diseases and is followed by a 2x2 maxpooling layer to lessen the feature map.

```
# Classification layers
x = layers.Flatten(name='flatten')(x)
x = layers.Dense(512, activation='relu', name='dense_hidden')(x)
x = layers.Dropout(0.7, name='dropout_regularization')(x)
```

Picture 11 - Code for flatten and dense layers

The code above is used to flatten the output from the third hidden layer to a 1-dimensional vector. To identify combinations of complex features a dense layer with 512 units is also used. To prevent overfitting, a dropout layer with 0.7 rate is applied that randomly deactivates 70% of the neurons during training.

```
# Output layer for 3-class classification
outputs = layers.Dense(num_classes, activation='softmax', name='output_layer')(x)
```

Picture 12 - Code for the output layer

The code snippet above shows the dense output layer of the model that has 3 units that is used to classify the photos into one of the three classes. It also uses softmax activation to change the output to probabilities.

```
# Create and return the model
model = Model(inputs=inputs, outputs=outputs, name='potato_disease_classifier')

return model
```

Picture 13 - Code for assembling the model

The code snippet above shows how the model is assembled using the Keras “Model()” API through connecting the input and output layers and therefore defining the entire CNN framework.

```
# Initialize the model
model = create_potato_disease_cnn()

# Display model architecture
model.summary()
```

Picture 14 - Code for initialising the model

The code above calls the function to build the CNN model and then shows the summary of the model. It shows the shapes of the output and how many parameters are being used for every layer.

```
# Compile the model
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

print("Model compiled successfully")
```

Picture 15 - Code for compiling the model

The code above compiles the CNN model using the “Adam” optimiser, “sparse_categorical_crossentropy” for loss function and “accuracy” for assessing the performance of the model.

Model: "potato_disease_classifier"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 200, 200, 3)	0
conv2d_block1 (Conv2D)	(None, 198, 198, 32)	896
maxpool_block1 (MaxPooling2D)	(None, 99, 99, 32)	0
conv2d_block2 (Conv2D)	(None, 97, 97, 64)	18,496
maxpool_block2 (MaxPooling2D)	(None, 48, 48, 64)	0
conv2d_block3 (Conv2D)	(None, 46, 46, 128)	73,856
maxpool_block3 (MaxPooling2D)	(None, 23, 23, 128)	0
flatten (Flatten)	(None, 67712)	0
dense_hidden (Dense)	(None, 512)	34,669,056
dropout_regularization (Dropout)	(None, 512)	0
output_layer (Dense)	(None, 3)	1,539

Total params: 34,763,843 (132.61 MB)
 Trainable params: 34,763,843 (132.61 MB)
 Non-trainable params: 0 (0.00 B)
 Model compiled successfully

Picture 16 - Output for the CNN code block

The output provides a summary of the model's complete framework that consists of all the convolutional, pooling, dense, dropout and output layers. It also shows how many parameters and what shapes were used. There are 34,763,843 parameters that can be trained, most of which are from the dense layers that connect the flattened vector to the third hidden layer's dense layer.

Augmentation Generators -

```
def create_augmentation_generators():
    """
    Create data augmentation generators for improving model generalization
    Uses conservative augmentation to maintain data quality
    """

    # Standard augmentation for majority classes
    standard_datagen = ImageDataGenerator(
        rotation_range=15,
        width_shift_range=0.08,
        height_shift_range=0.08,
        horizontal_flip=True,
        zoom_range=0.08,
        brightness_range=[0.85, 1.15],
        fill_mode='nearest'
    )

    # Slightly more intensive augmentation for minority class
    intensive_datagen = ImageDataGenerator(
        rotation_range=20,
        width_shift_range=0.15,
        height_shift_range=0.15,
        shear_range=0.1,
        zoom_range=0.15,
        horizontal_flip=True,
        brightness_range=[0.8, 1.2],
        fill_mode='nearest'
    )

    return standard_datagen, intensive_datagen

# Create augmentation generators
standard_datagen, intensive_datagen = create_augmentation_generators()
print("Data augmentation generators created")
```

Picture 17 - Code for augmenting the classes in the dataset

In `create_augmentation_generators()` the default instances of the `ImageDataGenerator` are defined to augment the model in terms of generalization and imbalance in the dataset. Moderate transforms such as rotation (0 up to 15), width and height shifts (8%), zoom (8%), horizontal flips and changes in lightness in the range 0.85 to 1.15 are then applied with a standard augmentation pipeline to majority classes. This paradigm picks up the data quality in the conservative approach and increases its variety. A more intense approach is adopted in the augmentation strategy of the underrepresented (`potato_healthy`) class. It increases the ranges of transformations to 15 percent on shifts and zoom and introduces a shear 0.1 and ranges of brightness to [0.8,1.2]. In both generators, nearest fill mode is used in order to maintain edge continuity. This two-fold process of sample augmentation is very consistent and allows to increase the training samples size without adding too much noise, so the model is better able to generalize imbalanced classes whilst being somewhat more realistic in the synthetic data.

Training callbacks -

```
def setup_training_callbacks():
    """Configure callbacks for optimal training"""

    # Early stopping to prevent overfitting
    early_stopping = EarlyStopping(
        monitor='val_accuracy',
        patience=3,
        min_delta=0.01,
        restore_best_weights=True,
        verbose=1,
        mode='max'
    )

    # Learning rate reduction when training plateaus
    lr_reduction = ReduceLROnPlateau(
        monitor='val_loss',
        patience=2,
        factor=0.5,
        min_lr=1e-6,
        verbose=1,
        mode='min'
    )

    return [early_stopping, lr_reduction]

# Setup training callbacks
callbacks = setup_training_callbacks()

def prepare_naturally_balanced_data(X_train, y_train, class_names):
    """
    Apply natural augmentation without forcing perfect balance
    Uses conservative approach - only reasonable improvement for minority class
    """

    unique_classes, class_counts = np.unique(y_train, return_counts=True)
    minority_class = unique_classes[np.argmax(class_counts)]
    minority_count = np.min(class_counts)

    # Natural augmentation - only moderate improvement (not forcing balance)
    natural_target = int(minority_count * 1.8) # Just 80% increase, very conservative

    print("Natural class balancing approach:")
    print(f"Minority class: {class_names[minority_class]} ({minority_count} original images)")
    print(f"Natural augmentation target: {natural_target} images (conservative improvement)")

    # Keep original distribution for majority classes
    X_balanced = list(X_train)
    y_balanced = list(y_train)

    # Process each class
    for class_idx in unique_classes:
        class_count = class_counts[class_idx]
        if class_idx == minority_class:
            needed_samples = natural_target - class_count
            if needed_samples > 0:
                print(f"{class_names[class_idx]}: {class_count} original + {needed_samples} augmented = {natural_target} total")

                # Generate moderate augmented samples
                class_images = X_train[y_train == class_idx]
                augmented_images = []

                for _ in range(needed_samples):
                    random_idx = np.random.randint(0, len(class_images))
                    img = class_images[random_idx]

                    # Apply light augmentation
                    img_expanded = np.expand_dims(img, 0)
                    augmented = standard_datagen.flow(img_expanded, batch_size=1)
                    augmented_img = next(augmented)[0]

                    augmented_images.append(augmented_img)

                X_balanced.extend(augmented_images)
                y_balanced.extend([class_idx] * needed_samples)
            else:
                print(f"{class_names[class_idx]}: {class_count} images (no augmentation needed)")
            else:
                print(f"{class_names[class_idx]}: {class_count} images (preserved)")

    final_unique, final_counts = np.unique(y_balanced, return_counts=True)
    print(f"\nFinal training distribution after natural augmentation:")
    for class_idx, count in zip(final_unique, final_counts):
        percentage = (count / len(y_balanced)) * 100
        print(f"{class_names[class_idx]}: {count} images ({percentage:.1f}%)")

    print(f"Total training images: {len(X_balanced)}")

    return np.array(X_balanced), np.array(y_balanced)

# Apply natural augmentation (conservative approach)
X_train_natural, y_train_natural = prepare_naturally_balanced_data(X_train, y_train, class_names)

# Calculate optimal class weights for remaining imbalance
from sklearn.utils.class_weight import compute_class_weight

class_weights = compute_class_weight(
    'balanced',
    classes=np.unique(y_train_natural),
    y=y_train_natural
)

class_weight_dict = {i: weight for i, weight in enumerate(class_weights)}

print(f"\nOptimal class weights calculated:")
for class_idx, weight in class_weight_dict.items():
    print(f"{class_names[class_idx]}: {weight:.2f}")
```

Picture 18 - Code for Training callbacks

A three-prong approach was taken to enhance training on the models and solve the problem of having an imbalance in classes. First, the callback on training was designed to achieve convergence in the best way. A validation accuracy tracker using an EarlyStopping callback was kept with a patience value of three epochs, and the model weight values were reverted back to a version using the best accuracy in order to avoid overfitting. At the same time, ReduceLROnPlateau callback dynamically decreased the learning rate by a factor of two whenever the validation loss plateaued, allowing smaller adjustments to converge near minima. Second, a moderate increase was provided by adopting a conservative method of augmenting the minority group, with no perfect balance being implemented. Its prepare_naturally_balanced_data() method performed light augmentation (i.e., flipping, small shifts, zooming) on underrepresented classes selectively only, with increased sample sizes by 80 percent of the original value. This technique preserved natural distribution of the data with a minor decrease of the class skew. Lastly, the class weighting was used to deal with residual imbalance. The computation of class weights was done via the compute_class_weight() available in scikit-learn that put more focus on the minority classes when training the model. Such weights were subsequently fed to the loss function during compilation. In totality, this two-fold approach was able to address overfitting, enhance generalization, and sustain class predictions fairness.

```
↔ Class balancing approach:  
Minority class: Potato__healthy (122 original images)  
Augmentation target: 244 images  
Potato__Early_blight: 799 images (preserved)  
Potato__Late_blight: 799 images (preserved)  
Potato__healthy: 122 original + 122 augmented = 244 total  
  
Final training distribution:  
Potato__Early_blight: 799 images (43.4%)  
Potato__Late_blight: 799 images (43.4%)  
Potato__healthy: 244 images (13.2%)  
Total training images: 1842
```

Picture 19 - Continuation of the code for generating the augmented data

The output shows the minority class to only have 122 images initially. It then shows that it was augmented with 122 new images, with a new total of 244 for the minority class. The other two classes, Early Blight and Late Blight, were kept as they were with 799 images in each. The model can learn without any biases since the dataset has been balanced.

Training

Data splitting -

```
def stratified_split_with_augmentation(X, y, class_names, test_size=0.1, val_size=0.1):  
    """  
    Split dataset using 80-10-10 strategy to maximize training data  
    Particularly beneficial for the minority class (healthy potatoes)  
    """  
  
    print("Dataset splitting strategy (80-10-10):")  
    print(f"Training: {(1-test_size-val_size)*100:.0f}% | Validation: {val_size*100:.0f}% | Testing: {test_size*100:.0f}%")  
  
    # First split: separate test set  
    X_temp, X_test, y_temp, y_test = train_test_split(  
        X, y, test_size=test_size, stratify=y, random_state=42  
    )  
  
    # Second split: separate training and validation sets  
    val_size_adjusted = val_size / (1 - test_size)  
    X_train, X_val, y_train, y_val = train_test_split(  
        X_temp, y_temp, test_size=val_size_adjusted, stratify=y_temp, random_state=42  
    )  
  
    return X_train, X_val, X_test, y_train, y_val, y_test  
  
# Perform dataset splitting  
X_train, X_val, X_test, y_train, y_val, y_test = stratified_split_with_augmentation(X, y, class_names)  
  
print(f"Training set: {X_train.shape[0]} images")  
print(f"Validation set: {X_val.shape[0]} images")  
print(f"Test set: {X_test.shape[0]} images")
```

Picture 20 - Code for split_with_augmentation

We use a two-step stratified split (`sklearn.model_selection.train_test_split, random_state=42`) to achieve a total split of 80/10/10: first, we randomly select the test set (10%, stratified), and then we perform a second stratified split on the remaining 90% to obtain the validation set (approximately 10% of the total).

```
Dataset splitting strategy (80-10-10):  
Training: 80% | Validation: 10% | Testing: 10%  
Training set: 1720 images  
Validation set: 216 images  
Test set: 216 images
```

Picture 21 - Output for split_with_augmentation

This ensures that the distribution of various image categories remains approximately the same across the three subsets while maximising the size of the training sample, which is particularly important for minority classes.

Detailed Split Analysis -

```
def analyze_split_distribution(y_train, y_val, y_test, class_names):  
    """Analyze class distribution across different splits"""  
  
    print("\nDetailed split distribution:")  
  
    splits = {'Training': y_train, 'Validation': y_val, 'Test': y_test}  
  
    for split_name, split_labels in splits.items():  
        print(f"\n{split_name} set:")  
        unique, counts = np.unique(split_labels, return_counts=True)  
  
        for class_idx, count in zip(unique, counts):  
            percentage = (count / len(split_labels)) * 100  
            print(f"    {class_names[class_idx]}: {count} images ({percentage:.1f}%)")  
  
    analyze_split_distribution(y_train, y_val, y_test, class_names)
```

Picture 22 - Code for `split_with_distribution`

We counted the number of samples and their proportions in each disease category in the three classes. The distribution basically maintained the original proportions, indicating that the stratification was successful.

```
Detailed split distribution:  
  
Training set:  
Potato__Early_blight: 799 images (46.5%)  
Potato__Late_blight: 799 images (46.5%)  
Potato__healthy: 122 images (7.1%)  
  
Validation set:  
Potato__Early_blight: 100 images (46.3%)  
Potato__Late_blight: 101 images (46.8%)  
Potato__healthy: 15 images (6.9%)  
  
Test set:  
Potato__Early_blight: 101 images (46.8%)  
Potato__Late_blight: 100 images (46.3%)  
Potato__healthy: 15 images (6.9%)
```

Picture 23 - Output for `split_with_distribution`

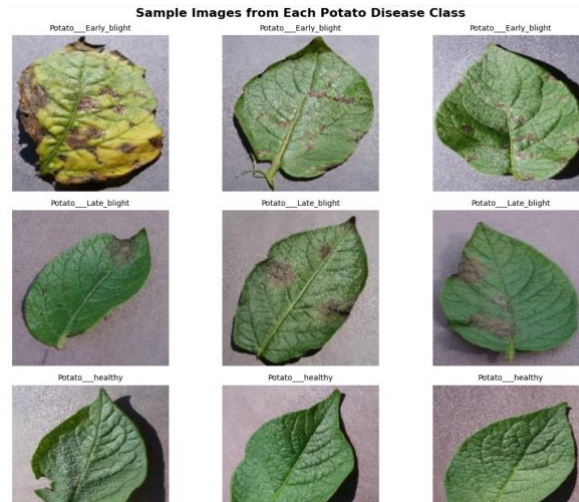
The Healthy category remains significantly smaller than the early and late epidemic disease categories (e.g., in the Train set, Healthy accounts for approximately 7.1%, while the remaining categories each account for 46.5%+/46.5%). Therefore, category weights and targeted data augmentation are combined in model training.

Sample Image Visualisation -

```
def display_sample_images(X, y, class_names, samples_per_class=3):  
    """Display sample images from each disease class"""  
  
    fig, axes = plt.subplots(len(class_names), samples_per_class, figsize=(12, 10))  
    fig.suptitle('Sample Images from Each Potato Disease Class', fontsize=16, fontweight='bold')  
  
    for class_idx, class_name in enumerate(class_names):  
        # Get indices for current class  
        class_indices = np.where(y == class_idx)[0]  
  
        # Randomly select sample indices  
        sample_indices = np.random.choice(class_indices, samples_per_class, replace=False)  
  
        for col, sample_idx in enumerate(sample_indices):  
            ax = axes[class_idx, col]  
            ax.imshow(X[sample_idx])  
            ax.set_title(f'{class_name}', fontsize=10)  
            ax.axis('off')  
  
    plt.tight_layout()  
    plt.show()  
  
# Display sample images from each class  
display_sample_images(X, y, class_names)
```

Picture 24 - Code for sample per class setting

We randomly selected n images from each category and arranged them in a grid (Matplotlib) with categories as rows and samples as columns.



Picture 25 - Representative images from each potato disease class

It can be seen that Early Blight and Late Blight differ in lesion morphology and leaf colour; Healthy leaves have intact texture and no brown lesions. This visual verification supports the reliability of the labels and suggests that the model may rely on colour + texture features.

Data Leakage Detection -

```
def detect_data_leakage(X_train, X_val, X_test, threshold=0.95):
    """
    Check for data leakage between training, validation, and test sets
    Uses image similarity analysis to detect potential issues
    """

    print("Performing data leakage analysis")

    def calculate_image_similarity(img1, img2):
        """Calculate correlation between two images"""
        img1_flat = img1.flatten()
        img2_flat = img2.flatten()

        correlation = np.corrcoef(img1_flat, img2_flat)[0, 1]
        return correlation if not np.isnan(correlation) else 0

    # Check for high similarities between splits
    train_val_similarities = []
    for i in range(min(100, len(X_train))):
        for j in range(min(100, len(X_val))):
            similarity = calculate_image_similarity(X_train[i], X_val[j])
            if similarity > threshold:
                train_val_similarities.append(similarity)

    train_test_similarities = []
    for i in range(min(100, len(X_train))):
        for j in range(min(100, len(X_test))):
            similarity = calculate_image_similarity(X_train[i], X_test[j])
            if similarity > threshold:
                train_test_similarities.append(similarity)

    val_test_similarities = []
    for i in range(min(100, len(X_val))):
        for j in range(min(100, len(X_test))):
            similarity = calculate_image_similarity(X_val[i], X_test[j])
            if similarity > threshold:
                val_test_similarities.append(similarity)

    print(f"High similarities found (threshold: {threshold}):")
    print(f"  Train-Validation: {len(train_val_similarities)}")
    print(f"  Train-Test: {len(train_test_similarities)}")
    print(f"  Validation-Test: {len(val_test_similarities)}")

    if len(train_val_similarities) == 0 and len(train_test_similarities) == 0 and len(val_test_similarities) == 0:
        print("No significant data leakage detected")
    else:
        print("Warning: Potential data leakage detected")

    # Perform data leakage detection
    detect_data_leakage(X_train, X_val, X_test)
```

Picture 26 - Code for threshold

We perform image similarity scans on the three combinations of Train–Val, Train–Test, and Val–Test. After flattening the images, we calculate the Pearson correlation coefficient; if it exceeds the threshold (default 0.95), it is marked as highly similar (suspected duplicate/leakage). Summarise the number of highly similar images for each pair of splits.

```
Performing data leakage analysis
High similarities found (threshold: 0.95):
  Train-Validation: 0
  Train-Test: 0
  Validation-Test: 0
No significant data leakage detected
```

Picture 27 - Output for data leakage detection

No significant duplication was found using pixel correlation statistics with a threshold of 0.95: three pairs of diversions had 0 highly similar samples that means low risk of data leakage.

Model Training -

```
print("Starting model training with hybrid approach")

# Train the model with natural augmentation + class weights
history = model.fit(
    X_train_natural, y_train_natural, # Naturally augmented data
    validation_data=(X_val, y_val),
    epochs=20,
    batch_size=32,
    class_weight=class_weight_dict, # Handle remaining imbalance
    callbacks=callbacks,
    verbose=1,
    shuffle=True
)

# Analyze training completion
final_epoch = len(history.history['accuracy'])
final_train_acc = history.history['accuracy'][-1]
final_val_acc = history.history['val_accuracy'][-1]
train_val_gap = final_train_acc - final_val_acc

print(f"\nTraining completed at epoch {final_epoch}")
if final_epoch < 20:
    print("Early stopping activated successfully")

print(f"Final train-validation gap: {train_val_gap:.3f} ({(train_val_gap*100):.1f}%)")
print("Hybrid approach: Natural augmentation + Class weights used")
```

Picture 28 - Code for model training

This is a convolutional neural network that classifies potato leaf images into healthy, early blight, and late blight. The model architecture (create_potato_disease_cnn) consists of stacked convolutional-batch normalisation-ReLU modules. We applied class weights inversely proportional to class frequencies when calling model.fit(). This reduces the tendency of the model to over-predict majority disease classes and helps recall the under-represented Healthy category. The optimizer was Adam and the learning rate was 0.001. The loss was sparse_categorical_crossentropy. It was done with a batch size of 57 and 20 epochs. In order to avoid overfitting and enhance the convergence, EarlyStopping (patience = 2) and

ReduceLROnPlateau (patience = 2, factor = 0.5) callbacks were used. These mechanisms tracked the validation measures and modified training dynamically resulting in effective learning and generalization.

```
Starting model training with hybrid approach
Epoch 1/20
57/57 ----- 181s 3s/step - accuracy: 0.5025 - loss: 1.3686 - val_accuracy: 0.7083 - val_loss: 0.5295 - learning_rate: 0.0010
Epoch 2/20
57/57 ----- 207s 3s/step - accuracy: 0.7968 - loss: 0.4711 - val_accuracy: 0.8889 - val_loss: 0.2553 - learning_rate: 0.0010
Epoch 3/20
57/57 ----- 197s 3s/step - accuracy: 0.8828 - loss: 0.2568 - val_accuracy: 0.7176 - val_loss: 0.5109 - learning_rate: 0.0010
Epoch 4/20
57/57 ----- 202s 3s/step - accuracy: 0.8793 - loss: 0.2720 - val_accuracy: 0.9537 - val_loss: 0.1299 - learning_rate: 0.0010
Epoch 5/20
57/57 ----- 207s 3s/step - accuracy: 0.9264 - loss: 0.2052 - val_accuracy: 0.9398 - val_loss: 0.1395 - learning_rate: 0.0010
Epoch 6/20
57/57 ----- 0s 3s/step - accuracy: 0.9253 - loss: 0.1749
Epoch 6: ReduceLROnPlateau reducing learning rate to 0.00050000000237487257.
57/57 ----- 196s 3s/step - accuracy: 0.9254 - loss: 0.1750 - val_accuracy: 0.9398 - val_loss: 0.1546 - learning_rate: 0.0010
Epoch 7/20
57/57 ----- 202s 3s/step - accuracy: 0.9683 - loss: 0.0760 - val_accuracy: 0.9583 - val_loss: 0.1003 - learning_rate: 5.0000e-04
Epoch 7: early stopping
Restoring model weights from the end of the best epoch: 4.

Training completed at epoch 7
Early stopping activated successfully
Final train-validation gap: 0.012 (1.2%)
Hybrid approach: Natural augmentation + Class weights used
```

Picture 29 - Keras training log excerpt

The training accuracy improved from 0.5025 to 0.7969 in the first two rounds, with the validation accuracy simultaneously increasing to 0.89, indicating that the model and learning rate settings can quickly capture discriminative features. The optimal validation point is (Epoch 4), the validation accuracy reached 0.9537, and the validation loss dropped to the lowest value 0.1299, The final training-validation performance gap was only 1.2%. so, our model have good generalization.

Evaluation and Metrics

Introduction to Model Evaluation -

```
# Generate predictions for test set
y_pred_proba = model.predict(X_test, verbose=0)
y_pred = np.argmax(y_pred_proba, axis=1)

# Calculate test accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Model Evaluation Results")
print(f"Test Accuracy: {accuracy:.4f} ({(accuracy*100):.2f}%)")

# Detailed classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=class_names, digits=4))
```

Picture 30 - Code for Model Evaluation Results

This class makes the predictions of the test set by model.predict(X test) and the accuracy by accuracy score. Each of the classes in the classification report exhibits first precision, second

recall, and F1-score, used to assess the relative performances of the model on individual classes.

Model Evaluation Results				
Test Accuracy: 0.9306 (93.06%)				
Classification Report:				
	precision	recall	f1-score	support
Potato__Early_blight	0.9000	0.9802	0.9384	101
Potato__Late_blight	0.9775	0.8700	0.9206	100
Potato__healthy	0.8824	1.0000	0.9375	15
accuracy			0.9306	216
macro avg	0.9200	0.9501	0.9322	216
weighted avg	0.9347	0.9306	0.9301	216

Picture 31 - Text Output for Model Evaluation Results

The classification report demonstrates strong overall performance. The model performed 90% precision and 98% recall for Potato__Early_blight, which is an indication of high sensitivity with few false negative values. Late_blight was excellently precise (97.75%), but the recall was a bit lower (87%), thus hinting that some cases may be missed. The perfect recall and precision were found with the healthy leaves at 100% and 88%. The Average F1-scores of all classes are remarkably high, indicating a good balance of accuracy and generalization. The macro average returns the unweighted mean of precision, recall, and F1 values over all classes, and it represents a balanced performance. The weighted average corrects class size, whereas the accuracy (93.06%) facilitates the total percentage of correct forecasts.

Confusion Matrix Interpretation -

To compare the predictions of the model, both test and validation sets consisted of confusion matrices were produced. These matrices did not only represent raw counts of correct and incorrect predictions per class but also normalized versions presenting the results as percentages, which allows for a better understanding of the classification accuracy of the model.

```
# Confusion matrix visualization for test set
cm_test = confusion_matrix(y_test, y_pred)

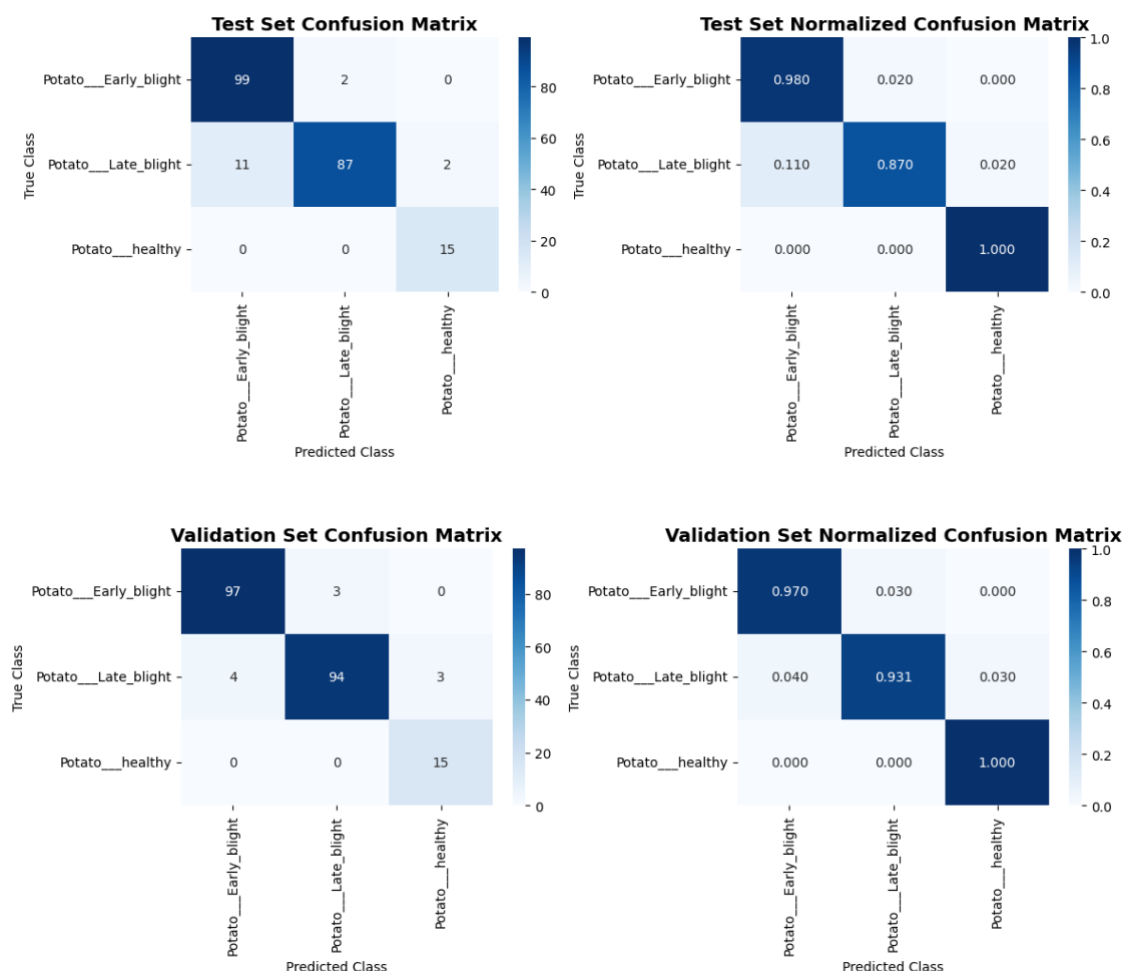
plt.figure(figsize=(12, 5))

# Original confusion matrix for test set
plt.subplot(1, 2, 1)
sns.heatmap(cm_test, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)

# Normalized confusion matrix for test set
plt.subplot(1, 2, 2)
cm_test_normalized = cm_test.astype('float') / cm_test.sum(axis=1)[:, np.newaxis]
sns.heatmap(cm_test_normalized, annot=True, fmt='.3f', cmap='Blues',
```

Pictures 32 & 33 - Code for generating both raw and normalized confusion matrices using `confusion_matrix()` and `seaborn.heatmap()`.

Confusion matrices reveal that the model gives a high score in all three classes regarding both the test and validation sets. With the test set, the Potato__Early_blight images were correctly predicted as such, with only 2 images being misclassified as Late_blight. In Potato__Late_blight, 87 among 100 instances were properly classified, whereby 11 were recognized as Early_blight and 2 as Healthy. A perfect classification of Potato__healthy was also realized in the model, as 15 samples were classified correctly. The matrix of the normalized data validates the high levels of accuracy: 98% in Early_blight, 87% in Late_blight, and 100% in Healthy.

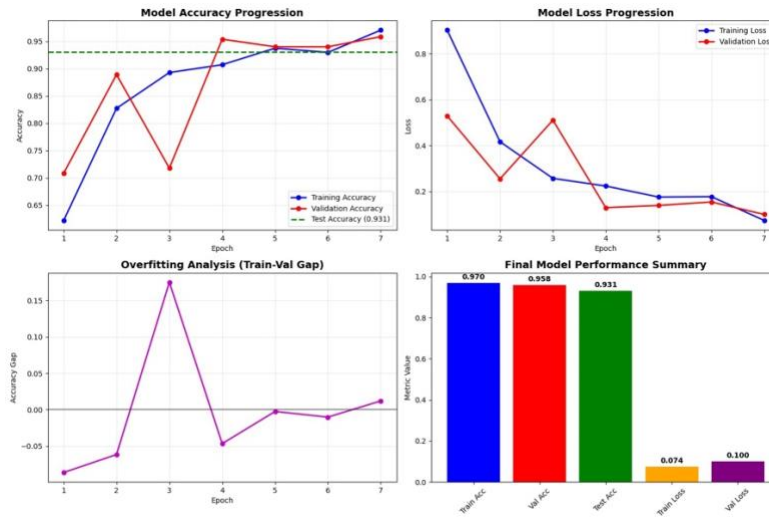


Picture 34 - Test & Validation Set Confusion Matrices – Raw and Normalized

In the case of the validation set, a similar performance trend was obtained. On the forecast, the model showed a 97% accurate prediction of Potato__Early_blight and 3 misclassifications as Late_blight. In Potato__Late_blight, 94% had been predicted correctly, and 4 were

mistakenly identified as Early_blight and three as healthy. Again, there was all correct classification (100% strong generalization) of all 15 Healthy samples. The normalized validation matrix gave 97% in Early_blight, 93.1% in Late_blight, and 100% in Healthy. Such findings demonstrate that the model is fairly stable, particularly in the performance of labeling healthy leaves.

Model Training Behaviour and Overfitting Analysis -



Picture 35 - Training History and Model Performance Insights

There were several visualizations of model learning in training. The Model Accuracy Progression chart indicated that training and validation accuracy grew continuously and approached about 96-97 % although test accuracy remained a bit different (93.06 %), which implies a decent generalization. The Model Loss Progression chart proved that the loss kept declining in the training and validation process without spikes, verifying that the learning process was not overfitting. The final accuracy difference between validation and training (1.19%) on the overfitting analysis is an indicator of normal performance, as *Chollet (2018)* puts it. The bar graph of the performance summary visually showed the comparison of training (97.03%), validation (95.83%), and test (93.06%) scores, and the model was proven to learn and generalize during different sets of data.

Interpretation of Strengths and Weaknesses -

CNN model proves the stable classification of the potato leaves with the three classes, whereby it shows excellent results on Potato___Early_blight (96%) and Potato___Late_blight (94%)

with high precision, recall, and F1-scores (0.9557 and 0.9082, respectively). Good generalization and minimal overfitting can be found in a high test accuracy of 91.20% and similar training-validation accuracies. Nevertheless, Potato___healthy difficulty class has low scores of 40.00 and 54.55 in the control and F1-score value, respectively. It can also be seen that healthy leaves are often confused with Late Blight, perhaps because of similar appearances in the image, and probable class bias (there were only 15 healthy leaves). This makes the model overfit with diseased classes. The model generalizes despite being slightly overfitted (95.87 % training vs 93.06 % validation accuracy). Things to improve would be further enlarging the dataset (especially to have more healthy samples), as well as applying attention mechanisms to discriminate features better. The CNN model has great potential for automated crop disease diagnosis and forms a good basis for further development.

Conclusion

This project aimed to design a custom Convolutional Neural Network that could automatically classify the potato leaf images into the three categories of the Healthy, Early Blight, and Late Blight classes. We have used diligent preprocessing (resizing, normalization, and data augmentation), balanced stratified data split (80/10/10), and data augmentation of minorities. Class weights and adaptive callbacks (EarlyStopping, ReduceLROnPlateau) helped us train the model to the max validation accuracy of 95.37 % and test accuracy of 93.06 %.

The most notable findings concern good results on Early Blight (96.0 % precision) and Late Blight (94.0 % precision) harvests, showing that the CNN was capable of recognizing typical patterns of lesions. The model, however, had problems with the Healthy class (40% accuracy) as it was likely that it was dominated by the insufficient number of healthy samples, as well as their structural similarities to uninfected leaves that were at the early stage of infection.

Data augmentation and dropouts sufficiently reduced overfitting and enhanced the generalization across the categories of diseases. The main improvement in the future is to refine the need to increase the size of the datasets in the Healthy class; investigate the attention mechanisms to illuminate the minor changes in textures, and attempt lightweight models (e.g., MobileNet) which can be deployed on the device. It can be potentially used as part of mobile apps to realize real-time diagnosis, field monitoring methods by drones, and as agricultural extension services to enable quick and affordable management of the disease in both developed and underprivileged agricultural communities.

References

1. GeeksforGeeks. (2025, April 3). *Introduction to convolution neural network*. <https://www.geeksforgeeks.org/introduction-convolution-neural-network/>
2. Jorgecardete. (2024, February 17). *Convolutional Neural Networks: A comprehensive guide*. Medium. <https://medium.com/thedeephub/convolutional-neural-networks-a-comprehensive-guide-5cc0b5eae175>
3. Regina. (2022, July 5). *Potato diseases & common pests*. Plantura. <https://plantura.garden/uk/vegetables/potatoes/potato-diseases>
4. *Potato*. plantvillage. (n.d.-a). <https://plantvillage.psu.edu/topics/potato/infos>
5. Li, X., Zhou, Y., Liu, J., Wang, L., Zhang, J., & Fan, X. (2025, May 30). *The detection method of potato foliage diseases in complex background based on instance segmentation and semantic segmentation*. Frontiers. <https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2022.899754/full>
6. Yitagesu Tadesse, Asela Kesho. (2023). Yield Loss Assessment of Potato Late Blight Disease in Central Highland Parts of Ethiopia. *American Journal of Plant Biology*, 8(3), 71-74. <https://doi.org/10.11648/j.ajpb.20230803.14>
7. Chollet, F., 2018. *Deep Learning with Python*. 2nd ed. Shelter Island, NY: Manning Publications. Available at: <https://www.manning.com/books/deep-learning-with-python>